PROJET DAVIDLISNARD · ÉTAT AU 13 NOVEMBRE 2026

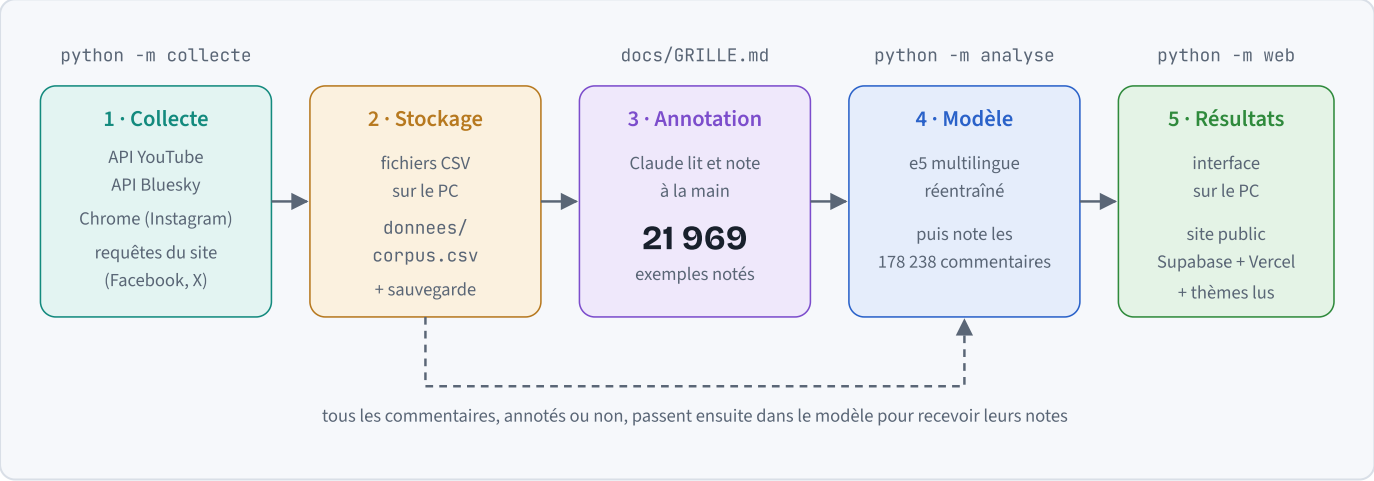
Comment marche l'analyse des commentaires

De la collecte des commentaires sur YouTube, Bluesky, Instagram, Facebook et X jusqu'au site public : d'où viennent les données, où elles sont rangées, comment Claude les annote et comment le modèle apprend à noter seul les 178 238 commentaires.

VUE D'ENSEMBLE

Le circuit, en cinq étapes

Chaque lancement de `python -m collecte` fait tourner tout le circuit pour ce qui est nouveau. Le modèle n'est réentraîné que sur demande, quand il y a beaucoup de nouvelles annotations.



ÉTAPE 1 · COLLECTE

D'où viennent les données

Chaque source a sa méthode : une API officielle quand il y en a une, sinon un navigateur que le programme pilote comme le ferait une personne. La collecte est incrémentale : un fichier `etat.json` par source note ce qui a déjà été lu, et chaque lancement ne va chercher que le nouveau.

API officielle

YouTube

71 632

commentaires et réponses. 22 chaînes suivies : ses chaînes et celles de ses soutiens en entier, les médias et commentateurs seulement sur les vidéos qui citent « Lisnard ». Et, au-delà, les vidéos de 82 autres chaînes trouvées par la recherche du mot-clé dans tout YouTube (9 563 commentaires).

API YouTube Data v3 (`commentThreads`, `search`) avec une clé gratuite. Le quota est d'environ 10 000 unités par jour ; une page de recherche de 50 vidéos en coûte 100, d'où 5 pages au plus par lancement : les vidéos récentes d'abord, puis l'arriéré.

API ouverte, sans compte

Bluesky

8 278

messages publics qui citent « Lisnard » et leurs réponses : tout le fil sous un message de tête qui le cite, et les réponses à une réponse qui le cite. Des comptes peuvent aussi être suivis en entier (aucun aujourd'hui).

API publique `api.bsky.app` (`searchPosts`, `getPostThread`), gratuite. Sans compte, la suite d'une recherche se demande par dates. Les messages de moins de 7 jours sont relus : nouvelles réponses et « j'aime ».

Navigateur piloté

Instagram

96 537

commentaires sous les posts de @davidlisnard.

Instagram n'a pas d'API publique pour ça. Un Chrome visible, piloté par Playwright et connecté avec un compte dédié (cookies enregistrés), ouvre les posts et déplie les réponses. On lit 100 posts par lancement et on ne revient que sur ceux de moins de 10 jours. La source s'arrête seule si Instagram freine.

Compte dédié, requêtes du site

Facebook

1 791

posts, commentaires et réponses de sa Page officielle et de 5 groupes publics de soutiens : « Cap 2027 avec David Lisnard », « Soutien David Lisnard 2027 », « Soutien à David Lisnard », « Soutien Lisnard Lot, Cantal et Aveyron », « Nouvelle Énergie 13 - Bouches-du-Rhône ». Les groupes privés sont écartés.

Meta a fermé l'API des groupes en 2024. Avec les cookies d'un compte dédié, la bibliothèque `facebook-gql-scraper` envoie les requêtes internes du site, sans navigateur. Rythme humain : 1,5 à 3,0 s entre deux requêtes, 120 min par lancement, 6 000 requêtes par jour ; le récent d'abord (7 jours), puis l'arriéré jusqu'au 31 juillet 2026. Au moindre freinage, la source se repose 3 h. Une Page se lit comme un profil, avec la même mécanique.

Compte dédié, requêtes du site

X (Twitter)

1 904

posts et réponses de 4 comptes suivis en entier (@davidlisnard, @Nouv_Energie, @AvecLisnard, @DavidLPresident), et les posts d'autres comptes qui citent « Lisnard », avec toutes les réponses dessous.

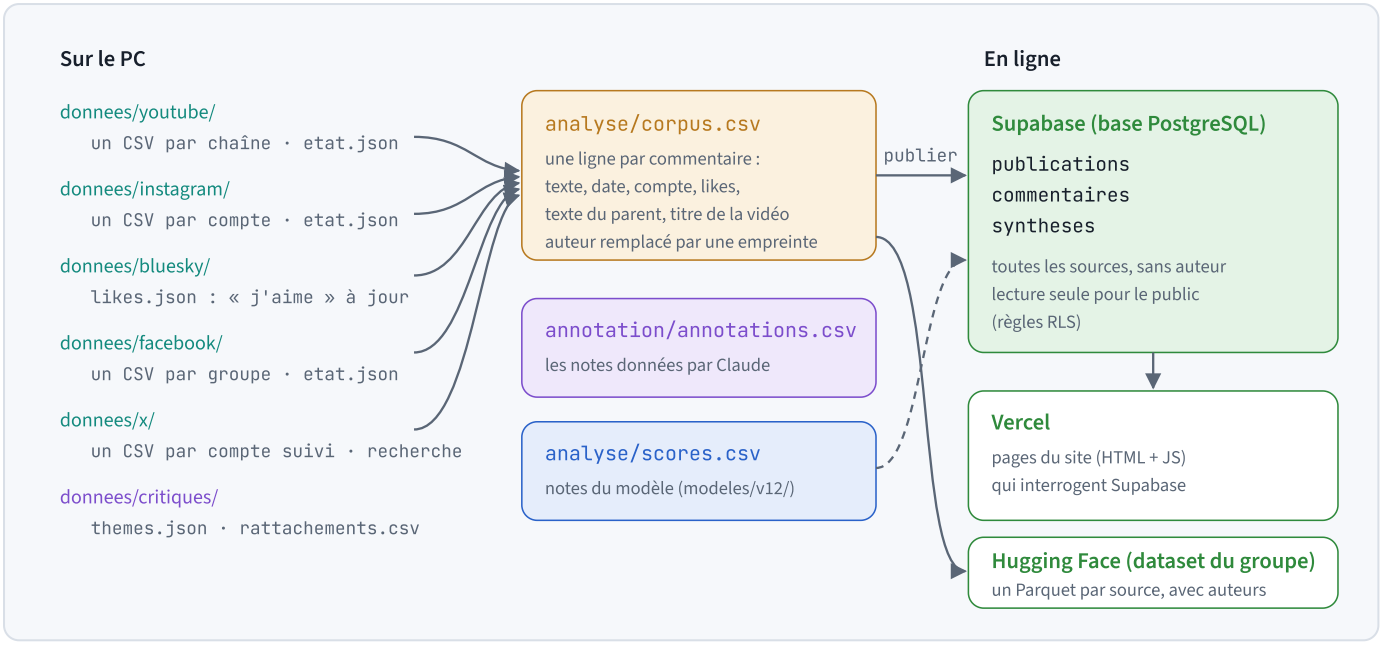
X n'a plus d'accès gratuit. Avec les cookies d'un compte dédié, la bibliothèque `twscrape` envoie les recherches internes du site, sans navigateur ; seule la recherche « conversation_id: » rend toutes les réponses d'un post (544 contre 36 en lisant le post, essai du 12 novembre 2026). Les petits fils sont lus par paquets de dix. Rythme : 40 recherches au plus par quart d'heure (X en permet 50), 3 à 8 s entre deux requêtes, 120 min par lancement, 3 000 requêtes par jour ; le récent d'abord (les posts suivis de moins de 7 jours sont relus, ceux qui le citent le sont au bout de 2 jours), puis l'arriéré jusqu'au 31 juillet 2026. Au moindre freinage, repos de 3 h. La connexion se fait une fois dans un Chrome visible (X refuse le mot de passe envoyé par un script).

> Écartés : TikTok (le contournement de sa protection anti-robots a été refusé), Reddit (bloqué par le réseau), WhatsApp (la discussion d'adhérents collectée jusqu'au 12 novembre 2026 : des échanges entre militants, pas l'avis du public ; retirée avec tout ce qui en venait). Threads demande une connexion : à tenter plus tard, avec un compte créé pour la collecte.

ÉTAPE 2 · STOCKAGE

Où et comment c'est rangé

Tout est d'abord stocké sur le PC, dans des fichiers lisibles (CSV, ouvrables dans Excel, et JSON). Il n'y a pas de base de données locale. Seul ce qui est publiable part ensuite en ligne.



La publication est incrémentale : `donnees/web/publie.json` retient ce qui est déjà en ligne, et seules les lignes nouvelles ou dont la note a changé sont envoyées. Les commentaires notés partent aussi vers le dataset du groupe RAG politique, `rag-politique/politique-sources`, dans `data/commentaires/nouvelle-energie/` : un seul fichier Parquet par source, avec le nom affiché des auteurs, remplacé en entier quand la source change (168 012 commentaires dans ces fichiers, dont 168 012 en attente d'envoi ; le site n'en dépend pas ; règles dans `docs/HUGGINGFACE.md`).

DOUBLONS

10 226

commentaires mis de côté (5,7 %) : le même texte, du même auteur, dans la même source. Seul le premier publié compte dans la synthèse, sur le site et dans les thèmes. Le même texte écrit par des personnes différentes reste compté : ce sont autant de voix.

« J'AIME »

Un commentaire lu juste après sa publication en a peu. Quand une publication est relue, les nouveaux comptes vont dans `likes.json`, sans retoucher les CSV. Ils servent à la vue pondérée : chaque commentaire y pèse 1 + ses « j'aime ».

SAUVEGARDE

23 Mo

`donnees/sauvegardes/donnees.zip`, refaite à la fin de chaque collecte : CSV bruts (Instagram d'abord), annotations, thèmes. Une seule copie : la nouvelle est écrite à part, relue, puis remplace l'ancienne. Refusée si un CSV a rétréci.

ÉTAPE 3 · ANNOTATION

Ce qu'on note : cinq notes sur 20 et une cible

Chaque commentaire reçoit les mêmes six réponses, définies dans `docs/GRILLE.md`.

CRITÈRE	QUESTION POSÉE	ÉCHELLE
Cible	Le commentaire parle-t-il de Lisnard (L) ou d'autre chose (A) ?	L ou A
Tonalité	Quel est le ton envers lui, de l'hostilité franche à l'éloge ?	`0 hostile · 10 neutre · 20 très favorable`
Forme	Reproche ou éloge sur sa manière de parler, son style, sa personne.	`0 reproche · 10 non évoqué · 20 éloge`
Système	Reproche ou éloge sur ce qu'il représente : parti, élus, « système ».	`0 reproche · 10 non évoqué · 20 éloge`
Idées	Reproche ou éloge sur son programme, ses propositions.	`0 reproche · 10 non évoqué · 20 éloge`
Pertinence	Le commentaire apporte-t-il un avis argumenté ? Un « Bravo ! » seul vaut 8. Une ironie impossible à trancher vaut 2 ou 3.	`0 vide · 20 très argumenté`

Si la cible est A, le ton et les trois reproches valent 10 : le commentaire ne le concerne pas. La pertinence permet ensuite d'écarter le bruit, comme les émojis seuls ou le hors-sujet (pertinence inférieure à 5).

Qui annote : Claude, à la main, dans la conversation

Il n'y a ni annotateur humain ni appel automatique à une API. Le programme tire des lots de commentaires, et Claude les lit un par un, avec le message auquel ils répondent et le titre de la vidéo, puis écrit les six notes et une courte remarque. Une vraie ligne du lot 24 :

```
« tout à fait ! Pas entendu de discours aussi creux depuis longtemps 😂 »
→ répond à : « Pour une fois je ne vais pas voter contre quelqu'un mais plutôt pour quelqu'un.
Que la force soit avec vous, David. »
cible L · ton 3 · forme 10 · système 10 · idées 3 · pertinence 8
remarque : ironie : approuve pour dire discours creux
```

Ce qu'on annote

Des tirages au hasard, pour que le jeu représente le corpus, et des tirages ciblés : les cas où le modèle hésite, les réponses, les commentaires les plus visibles, les « suspects » dont la note contredit les mots, et des commentaires longs (40 mots ou plus) répartis entre les sources et les comptes, surtout ceux qui argumentent.

Constance

100 commentaires renotés à l'aveugle d'une session à l'autre : même cible dans 96 % des cas, 0,57 point d'écart moyen sur la tonalité. C'est la meilleure précision qu'on peut espérer du modèle.

Ironie et réponses

Une ironie comprise est notée selon son sens réel. Une ironie indécidable reçoit 10 partout et une pertinence de 2-3, pour pouvoir la filtrer. Une réponse est toujours lue avec son parent.

ÉTAPE 4 · MODÈLE

Comment le modèle apprend à noter seul

Annoter les 178 238 commentaires à la main est impossible. On apprend donc à un modèle à reproduire les notes de Claude à partir des 21 969 exemples, puis il note tout le reste.

Le modèle de départ

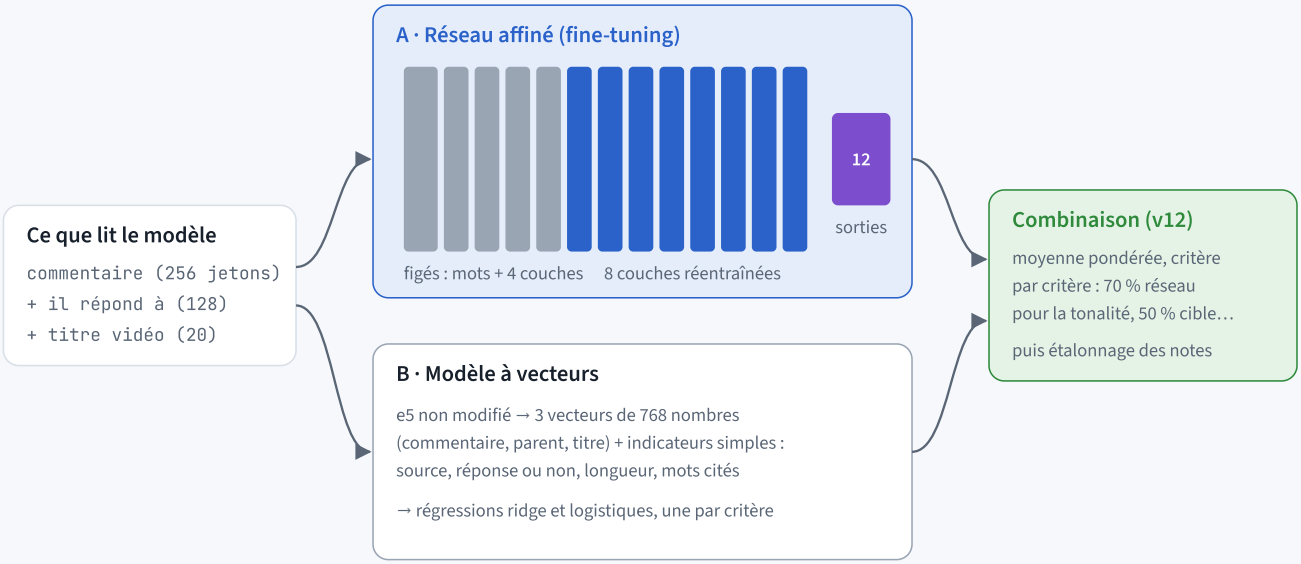
intfloat/multilingual-e5-base : un modèle de langue libre (famille BERT/XLM-RoBERTa, environ 280 millions de paramètres, 12 couches), entraîné par Microsoft pour transformer un texte en un vecteur de 768 nombres qui résume son sens. Il comprend le français et tourne sur le processeur, sans carte graphique. Seul, il ne sait rien de nos critères : il faut l'adapter. Il est lu sur le disque, sans rien demander en ligne (`HF_HUB_OFFLINE=1`, posé seul tant que sa copie existe).

Les annotations sont partagées en trois



Le modèle ne voit jamais les 489 commentaires de test pendant l'entraînement : les scores donnés plus bas sont donc ceux qu'il obtient sur du neuf.

Deux modèles, combinés



Un jeton est un morceau de mot : 256 jetons couvrent environ 180 mots, assez pour presque tous les commentaires (au-delà, la fin est ignorée). Le passage de 128 à 256 jetons date du 11 novembre 2026 (v11), pour que les longs commentaires argumentés soient lus en entier. Les 12 sorties du réseau : 1 pour la cible, 1 pour la tonalité, 1 pour la pertinence, et 3 classes (reproche, non évoqué, éloge) pour chacun des trois critères forme, système et idées.

Le fine-tuning, concrètement

1. On ajoute au bout d'e5 une petite couche de sortie qui produit les 12 réponses.
2. On fige les couches basses, qui comprennent les mots et la grammaire, et on laisse les 8 couches hautes s'ajuster à nos critères. C'est plus rapide, et 21 969 exemples ne suffiraient pas à bien régler tout le réseau.
3. Le réseau lit les 18 797 exemples par paquets de 16, compare ses notes à celles de Claude et corrige ses poids un peu à chaque paquet (taux d'apprentissage $3 \cdot 10^{-5}$ pour e5, 10^{-3} pour la couche de sortie). Il refait le tour complet 5 fois (5 époques).

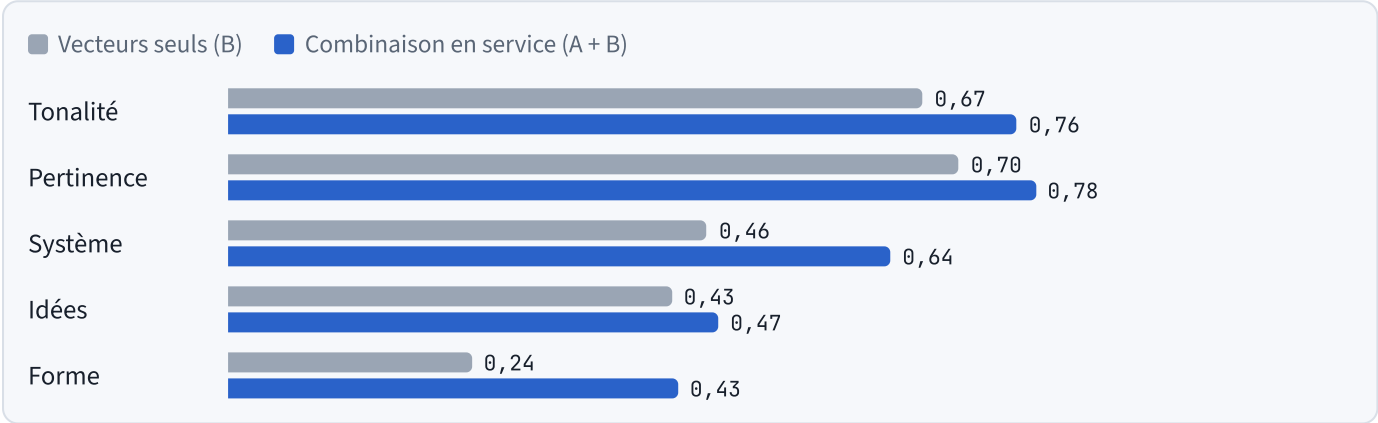
4. Sur le processeur, cela prend environ 8 h 47 (affine9, terminé le 12 novembre 2026 à 20:08).
5. On règle ensuite, sur les 2 563 exemples de réglage, le poids donné au réseau et aux vecteurs pour chaque critère. Le résultat est enregistré comme une nouvelle version (`modeles/v12/`).

Commande complète : `python -m analyse.ensemble tout`, aussi lancée par un bouton de l'interface. Puis `python -m analyse` renote tout le corpus, à 10 à 30 commentaires par seconde.

QUALITÉ MESURÉE

Ce que vaut le modèle

La corrélation de rang mesure si le modèle classe les commentaires dans le même ordre que Claude (1 = ordre identique, 0 = aucun lien). Mesures sur les 489 commentaires de test, version v12.



TONALITÉ

1,80 pt

d'écart moyen sur 20 avec la note de Claude. Entre deux lectures de Claude, l'écart est de 0,57.

CIBLE

83 %

de bonnes réponses sur « parle de Lisnard ou non ».

TON INVERSÉ

1,5 %

de critiques notées comme des éloges, ou l'inverse : 6 sur 401 commentaires de test qui parlent de lui (v12 ; 1,5 % avec v9). 0 sur 66 parmi les commentaires argumentés.

Les reproches de forme, système et idées sont rares (la plupart des commentaires n'en parlent pas), d'où des corrélations plus faibles. Les parts de critiques et d'éloges restent proches des vraies.

Quelle note est affichée

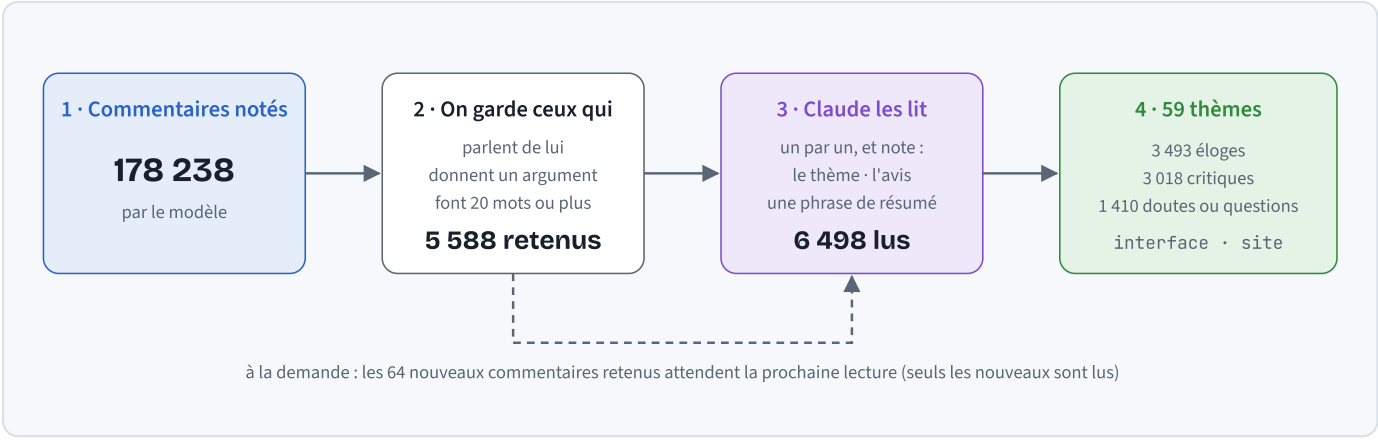
Correction à la main → sinon → Note de Claude (« relue ») → sinon → Note du modèle

Une note de Claude vaut aussi pour les copies exactes du même texte (60 caractères ou plus).

LECTURE PAR THÈMES

Ce qu'on lui reproche, ce qu'on salue

Les notes disent si un commentaire est favorable ou critique, pas sur quoi. Les commentaires argumentés sont donc lus par Claude et rangés par thème.



Commandes : `python -m critiques tirer`, `voir`, `noter`, puis `proches`, `a-resumer` (seules les synthèses qui ont reçu assez de nouveaux commentaires sont réécrites), `syntheses` et `python -m web publier`. Les comptes sont donnés en commentaires et en auteurs distincts. Ce n'est pas un sondage.

Cinq catégories pour les critiques et les doutes

Chaque critique ou doute rangé dans un thème porte aussi une catégorie : ce que le programme peut en faire. Les éloges n'en ont pas. Claude la pose en lisant, à la demande et seulement sur les commentaires déjà rangés par thème, jamais sur tout le corpus.

1 · OPPOSITION DE PRINCIPE

1 251

rejette le projet ou la personne en bloc ; aucune précision ne le fera changer d'avis.

2 · DEMANDE DE PRÉCISION

487

d'accord ou pas encore décidé, demande comment, combien, quand.

3 · OBJECTION SUR UNE MESURE

1 504

conteste un point précis (mesure, propos, façon de faire), souvent avec son cas personnel.

4 · DOUTE SUR LA FAISABILITÉ

516

ne conteste pas l'idée mais croit qu'elle sera bloquée ou qu'il n'y arrivera pas.

5 · SUGGESTION

670

apporte une idée ou un sujet absent du programme.

SANS CATÉGORIE

0

critiques ou doutes qui attendent le prochain tri.

Les nouveaux lots se notent `n | Thème | C3 | phrase` : chiffre obligatoire après C ou D, interdit après E. Pour l'existant : `python -m critiques trier` affiche le lot suivant sans catégorie, `python -m critiques ranger K fichier` l'enregistre (colonne `categorie` de `rattachements.csv`, lots dans `donnees/critiques/tris/`). L'onglet et la page Thèmes ont un filtre « Catégorie des critiques et des doutes » (Toutes, Tout sauf les oppositions de principe, ou une seule).

ÉTAPE 5 · RÉSULTATS

Où on lit les résultats

Interface sur le PC

`python -m interface` (Interface.bat) : lancer et planifier les collectes, régler chaque source et ajouter ce qu'elle suit par un simple lien (onglets YouTube, Bluesky, Instagram, Facebook, X), synthèse des notes, évolution par mois, exemples, qualité du modèle, thèmes. Les commentaires se filtrent par source, compte, période, notes et taille en mots, et se trient par la colonne « Mots ». La Synthèse peut se lire pondérée par les « j'aime ».

Site public

`analyse-commentaires-lisnard.vercel.app` : Le projet, Statistiques, Commentaires, Thèmes. Les filtres sont calculés dans le navigateur ou par une fonction SQL de Supabase, dont la taille en mots (colonne `mots` calculée par la base, tri « Plus longs »). Chaque page affiche la date de la dernière collecte et la version du modèle ; les Statistiques ont une vue pondérée par les « j'aime ». Ni auteurs ni doublons.

Thèmes

Onglet de l'interface et page du site : les thèmes filtrés par sujet, type d'avis, catégorie des critiques et des doutes, mot et, sur le site, par période (derniers mois ou mois choisis, avec la part de chaque thème comparée à la période précédente), avec leurs comptes, l'évolution par mois, les synthèses et chaque commentaire avec sa phrase, qui renvoie à l'original.

Un lancement, de bout en bout

1. `python -m collecte` lit le nouveau sur YouTube, Bluesky, Instagram, Facebook et X et l'ajoute aux CSV, puis refait la sauvegarde.
2. Le corpus est reconstruit (doublons marqués, « j'aime » à jour) et les nouveaux commentaires sont notés par le modèle en service.
3. S'il y a du neuf, `python -m web publier` envoie les lignes nouvelles à Supabase et le site les montre aussitôt, puis `python -m huggingface` remplace, dans le dataset Hugging Face du groupe, le fichier des sources qui ont changé (aussi par le bouton « Export Hugging Face » de l'interface).
4. De temps en temps : nouveaux lots annotés par Claude, puis réentraînement (nouvelle version du modèle) et renotation complète.
5. À la demande : lecture par Claude des commentaires argumentés restés sans thème, avec la catégorie des critiques et des doutes, puis les synthèses qui ont changé et la publication. L'ordre complet de ces étapes est dans `PROCESS.md`.

Document généré le 13 novembre 2026 par `python -m docs` depuis `docs/FONCTIONNEMENT.html`. Les chiffres viennent des données du projet à cette date.